Python Engineer Challenge — RAG Chatbot with FastAPI, Agno, NiceGUI

What We're Evaluating

- Your ability to learn quickly from excellent docs (Agno, NiceGUI, FastAPI)
- Simple, clean, maintainable code (not over/under-engineered)
- Useful, minimal tests at each step (unit + integration) that catch real bugs
- Thoughtful, minimal abstractions (see common patterns and extract them)
- Correct, robust real-time data handling (streaming; async, non-blocking UI)
- Maximally leverage libraries; do not reinvent what Agno/FastAPI/Pydantic already provide
- Practical, explicit Cursor usage: configured, leveraged, and documented

---

Constraints (Non-negotiable)

- Python 3.13+, asyncio, modern typing (e.g., `list[str]`, `str | None`), Pydantic models for structured data (no raw dicts)
- Agno for agent/orchestration; leverage Agno session/history features where applicable
- FastAPI for the backend runtime and streaming endpoint(s)
- NiceGUI used as a thin UI to visualize how the FastAPI backend is operating
- Streaming comes first: only after streaming works end-to-end, add upload
- Upload scope: one document type only — PDF
- OpenAI as the LLM; API key via `.env` (no secrets in code or git history)
- Cursor configuration is mandatory and must be documented in the README

Notes:

- If you add a thin layer on top of Agno (e.g., knowledge store or session glue), document why. Prefer using Agno's built-ins when possible.
- The FastAPI part will be reviewed more critically than the NiceGUI part; the UI is really just there to demonstrate the backend behavior.

What to Build

A minimal "Document QA Chatbot" that:

1) Streams agent responses token-by-token via a FastAPI endpoint
2) Lets a user upload a PDF, parses it, and makes it available to the agent
3) Shows async “thinking steps” in the UI without blocking streaming

Tools & Docs:

- Agno: https://docs.agno.com
- FastAPI: https://fastapi.tiangolo.com
- NiceGUI: https://nicegui.io/documentation

No long examples are provided. Minimal snippets below are only to disambiguate libraries and signatures.

Core Requirements (70 points)

1) Streaming Chatbot (25 points)

- FastAPI app with an endpoint that streams the agent’s response in chunks
- Agno agent configured with OpenAI; session/history leveraged when appropriate
- NiceGUI page displays streaming chunks as they arrive; preserves session chat history
- Errors surface as friendly UI messages; backend returns meaningful status codes
- Tests (mandatory):
    - Unit tests for configuration and agent wiring
    - Integration test that verifies multiple streamed chunks (not a single blob)

Minimal disambiguation (example signatures only):

```
# FastAPI streaming endpoint shape (do not copy verbatim; keep it minimal in your code)
from fastapi import FastAPI
from starlette.responses import StreamingResponse

app = FastAPI()

def token_generator(prompt: str):  # yield str tokens
    yield "..."

@app.get("/stream")
def stream(prompt: str):
    return StreamingResponse(token_generator(prompt), media_type="text/plain")

# Agno import surface (use OpenAI; configure via env)
from agno.agent import Agent
from agno.models.openai import OpenAI
```

□2) PDF Upload & Parsing (25 points)

- UI supports upload for: .pdf
- Parsing returns clean text and basic metadata; reject unsupported/oversized/empty files
- PDF content becomes available for the agent to use in responses (knowledge/memory)
- Prefer Agno mechanisms for knowledge/memory/history; if you add a thin layer, document why
- Tests (mandatory):
    - Unit tests for the PDF parser with real sample files in `tests/data`
    - Error tests (corrupt/empty/oversized)
    - Integration test: upload → parse → query → answer referencing the PDF

3) Async Status Signals (20 points)

- UI shows non-blocking status steps (e.g., Analyzing → Searching → Generating)
- Status clears on completion; errors display clearly
- Any additional async observability (progress, timing) is considered a plus

---

Code Quality (25 points)

- Architecture & Abstractions (10): minimal, obvious, DRY; rationale clearly documented
- Code Craftsmanship (8): modern typing; heavy use of Pydantic; readable naming; robust errors; sensible logging
- Testing & Documentation (7): tests are simple and meaningful; README explains setup, design choices, trade-offs

Reinventing functionality provided by Agno/FastAPI/Pydantic (without a documented reason) will be penalized.

Cursor Setup (Mandatory)

Your README must include a section explaining how you configured Cursor for this project:

- MCP servers (e.g., Agno docs, NiceGUI docs, FastAPI docs)
- Linters/formatters (ruff, black, ty/mypy; how errors are surfaced in the IDE)
- `.cursorrules` content and purpose (if used)
- Documentation indexing and any IDE features you enabled
- What actually helped you move faster and write simpler code

---

Testing Standards (Mandatory)

- Use pytest with `pytest_check` for assertions; use `pytest.raises` for expected errors
- Keep tests small and honest; do not hide failures with try/except or broad status acceptance
- No mocks in integration tests; prefer real flows and real sample files
- Place real PDFs in `tests/data`; ensure tests run deterministically

Deductions (examples):

- No/poor tests: −15
- Hiding failures (try/except, accepting 4xx/5xx as success): −8
- Hardcoded secrets: −10
- Old typing (`List`, `Optional`, `Union`): −3
- Missing README/setup or missing Cursor setup: −5 (each)

Deliverables

1) **Link to a public GitHub repository of the code:**
   – Source code organized by responsibility (backend/FastAPI, agent/Agno, UI/NiceGUI, parsing)
   – Tests in `tests/unit` and `tests/integration`, with `tests/data` sample PDFs
   – Configuration: `pyproject.toml` or `requirements.txt`, `.env.example`, `.gitignore` (exclude `.env`, logs, `__pycache__`)
   – README including:
     - Setup/run/test instructions
     - Architecture overview and abstractions (why they're the simplest that work)
     - Cursor configuration (mandatory)
     - Trade-offs, limitations, and what you'd add next
   – OpenAI provider must be configured via env vars in `.env` (e.g., `OPENAI_API_KEY`); do not commit secrets.

2) **Link to the app deployed in a cloud environment**

Evaluation Rubric (100 points)

- Core Requirements (70): Streaming (25), PDF Upload/Parsing (25), Async Status (20)

- Code Quality (25): Abstractions (10), Craftsmanship (8), Testing/Docs (7)
- Penalties applied as listed in Testing Standards and Deliverables

What stands out:

- Fast, simple, readable code that uses libraries effectively
- Clear separation between FastAPI (backend) and NiceGUI (demo UI)
- Thoughtful use of Agno features (session/history/knowledge) with rationale if extended
- Tests that fail loudly when something breaks and prove behavior
- README that shows how you configured Cursor and how you think

No unnecessary length; no long examples. Minimal, high-signal implementation that works end-to-end.

Bonus Points:

The completeness of the application is one of the criteria we will value with points. You can also get extra points for:

- Hosting the app on platforms on AWS with best practices
- API documentation
- Creativity - add anything you want to the app. It certainly won't hurt.